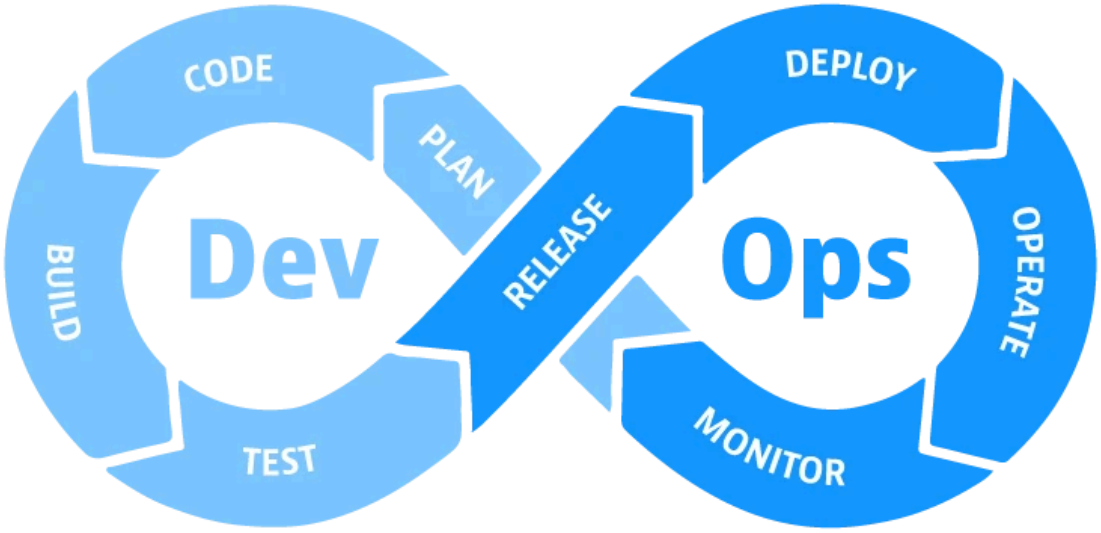
TP Integrador DevOps 2C 2026

Alumno: Santiago Abelle



Universidad de Palermo

1. Resumen **3**
1.1. Endpoints de la API 3
2. Arquitectura del pipeline **4**
2.1. Diagrama de flujo 4
2.2. Descripción de las etapas 4
3. Justificación técnica y decisiones de diseño **4**
3.1. Optimización de contenedores 4
3.2. Estrategia de integración 5
4. Aplicación de la filosofía DevOps **6**
4.1. Marco: CALMS y las Tres Formas 6
4.2. Primera Forma: flujo y consistencia 7
4.3. Segunda Forma: feedback rápido y Andon Cord 8
4.4. Tercera Forma: aprendizaje y simulación de fallos 10
5. Principios Lean: reducción de desperdicio **11**
6. Observabilidad **12**
6.1. Logs estructurados 12
6.2. Plataforma de monitoreo y métricas 13
6.3. Dashboard y Golden Signals 13
7. Conclusiones y mejoras **15**

1. Resumen

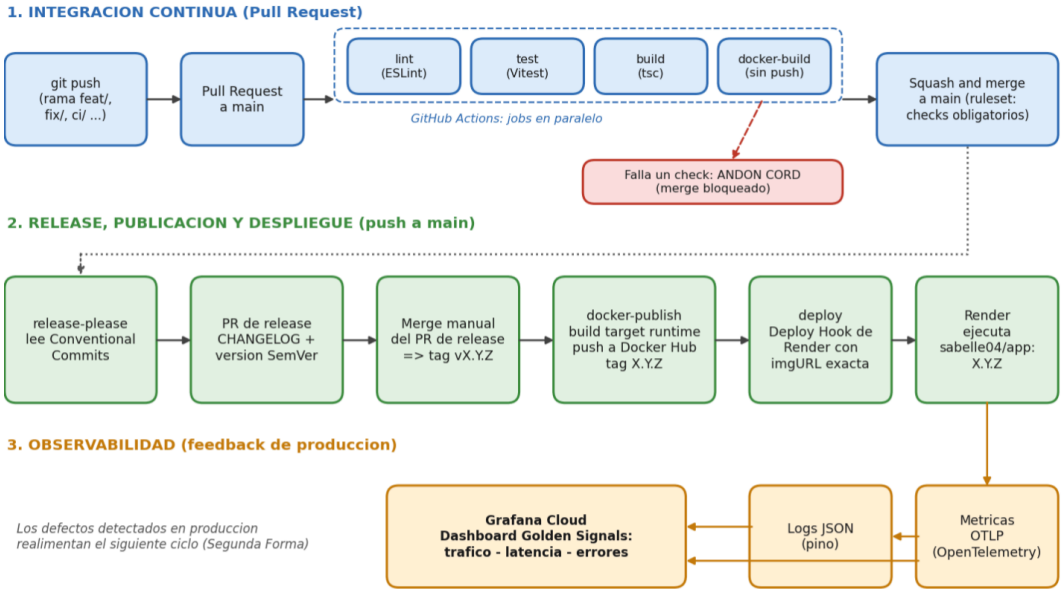
Este trabajo aplica la filosofía, las metodologías y las herramientas DevOps sobre el ciclo de vida completo de una API REST: desde el git push hasta el despliegue en producción y el monitoreo en tiempo real. La lógica de negocio es mínima (un CRUD de ítems, un health check y una ruta que falla a propósito) porque el foco está en el proceso: todo cambio llega a producción por un camino automatizado, verificado y trazable. El informe sigue la estructura pedida en la consigna y, en cada sección, relaciona las decisiones tomadas con los conceptos vistos en la cursada (CI/CD, Monitoreo y Observabilidad y Seguridad).

1.1. Endpoints de la API

Method	Ruta	Descripción
GET	/api/health	Healthcheck de la API
GET / POST	/api/items	Listar / crear items
GET / DELETE	/api/items/:id	Obtener / eliminar un item
GET	/api/error	Falla a propósito (HTTP 500) para probar el monitoreo
GET	/api/docs	Swagger UI
GET	/api/docs.json	Especificacion OpenAPI en JSON

2. Arquitectura del pipeline

2.1. Diagrama de flujo



2.2. Descripción de las etapas

- **Desarrollo:** Cada cambio se desarrolla en una rama corta (feat/, fix/, ci/, test/, build/, docs/) que parte de main y se integra mediante un PR con plantilla (descripción, tipo de cambio y evidencia de pruebas).
- **Build y Test (CI):** Al abrir o actualizar el PR, GitHub Actions ejecuta en paralelo lint (ESLint), test (Vitest + Supertest), build (compilación con tsc) y docker-build (construye la imagen sin publicarla). Un job inicial (changes) detecta si cambio código de app/ para no correr los checks cuando el PR no lo afecta.
- **Andon Cord:** El ruleset de main exige que los checks estén en verde. Si uno falla, el merge queda bloqueado.
- **Release:** Al hacer squash and merge, release-please lee los Conventional Commits y abre un PR de release con el CHANGELOG.md y la nueva versión SemVer. Se mergea el PR, lo que crea el tag vX.Y.Z y comienza el workflow Release.
- **Publicación de Imágen Docker y Deploy:** En el mismo workflow, el job docker-publish construye la etapa runtime del Dockerfile y la sube a Docker Hub con el tag SemVer. Luego el job deploy invoca el Deploy Hook de Render pasando la imagen con ese tag.
- **Observabilidad:** Render ejecuta la imágen. La aplicación emite logs JSON y exporta métricas por OTLP a Grafana Cloud, donde el dashboard permite detectar el estado de la app.

3. Justificación técnica y decisiones de diseño

3.1. Optimización de contenedores

El Dockerfile usa compilación multi-stage con cuatro etapas:

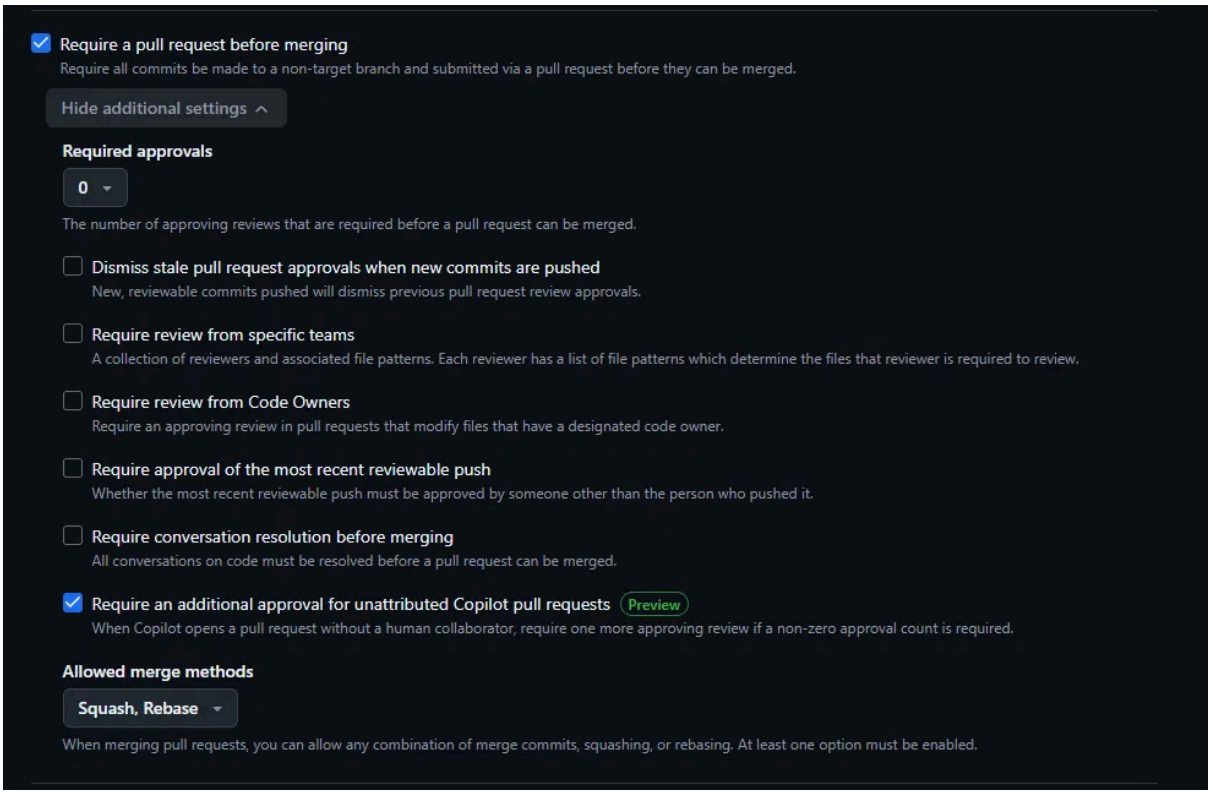
Etapa	Propósito
prod-deps	Instala solo las dependencias de producción (npm ci --omit=dev)
build	Instala todas las dependencias y compila TypeScript a JavaScript
dev	Entorno de desarrollo con recarga automática (tsx watch)
runtime	Imagen final: solo node_modules de producción, dist/ y package.json

- **Imagen base específica y liviana:** Elegí node:22.11.0-alpine3.20 porque es una imágen liviana y testeada que cubre las necesidades del proyecto.
- **Capas ordenadas para el cache:** Primero se copian package.json y package-lock.json y se instalan las dependencias; el código fuente se copia después. Cambiar código no reinstala dependencias (las capas de npm ci aparecen como CACHED). En CI el caché de capas se reutiliza entre ejecuciones.
- **Imagen final mínima:** No contiene TypeScript, devDependencies, tests ni código fuente.
- **Usuario sin privilegios:** Se usa el usuario non-root node.
- **.dockerignore:** Excluye node_modules, dist, .git, .env y documentacion del contexto de build.
- **Un solo Dockerfile para desarrollo y producción:** Solo cambia el target (dev o runtime).

3.2. Estrategia de integración

- **Modelo:** Desarrollo Trunk-Based basado en ramas efímeras por tipo de cambio que parten de main y se integran por PR en poco tiempo. Se descartó GitFlow por la complejidad de mantener ramas de larga vida (develop, release, hotfix), innecesaria para un proyecto tan mínimo.
- **Protección de main mediante un ruleset:** PR obligatorio (sin push directo), status checks obligatorios (lint, test, build, docker-build), rama actualizada antes de mergear, force push bloqueado y lista de bypass vacía.
- **Merge:** Squash and merge con el título del PR como mensaje. Cada PR deja un único commit convencional en main, que es lo que lee la herramienta de versionado.
- **Conventional Commits (feat:, fix:, docs:, ci:, test:, build:):** Además de ordenar el historial, permiten calcular la versión y generar el changelog automáticamente.
- **PR con template:** Plantilla en ***.github/pull_request_template.md*** con descripción, tipo de cambio y evidencia de pruebas.

- **Aprobaciones requeridas = 0:** El trabajo es individual y GitHub no permite aprobar el propio PR. La barrera de calidad es el CI obligatorio. Las ramas se eliminan automáticamente al mergear.



4. Aplicación de la filosofía DevOps

4.1. Marco: CALMS y las Tres Formas

Los materiales presentan los valores de DevOps como CAMS (Culture, Automation, Measurement, Sharing), extendidos a CALMS con Lean, y los principios operativos como Las Tres Formas. La tabla resume cómo aparece cada valor en el proyecto.

Valor	Representación
Culture	Responsabilidad compartida ("you build it, you run it"): quien escribe el cambio también lo despliega y lo monitorea. Cultura blameless: un check en rojo se corrige, no se esconde
Automation	Pipeline de punta a punta: verificación, versionado, build, publicación y despliegue sin pasos manuales (salvo la decisión de

	publicar)
Measurement	Métricas de sistema (Golden Signals) y logs estructurados; el dashboard muestra tráfico, latencia y errores en tiempo real
Sharing	Todo es visible: historial de PRs, checks públicos, changelog, imagen pública en Docker Hub, documentación de la API navegable
Lean	Lotes pequeños (un cambio por rama y PR) y eliminación de desperdicios

4.2. Primera Forma: flujo y consistencia

Las tareas manuales que quedaron 100 % automatizadas

Tarea	Antes (manual)	Ahora
Verificar lint, tests y compilación	Correr comandos localmente y confiar	Jobs de CI en cada PR
Verificar que la imagen construye	Probar docker build a mano	Job docker-build en cada PR
Calcular y asignar la version	Elegir un numero y crear el tag a mano	release-please desde los commits
Generar el changelog	Redactarlo a mano	Generado desde los Conventional Commits
Construir y publicar la imagen	docker build, tag, push	Job docker-publish tras el release
Desplegar	Entrar a la plataforma y actualizar	Job deploy con Deploy Hook y tag exacto

Entornos de ejecución idéntico entre desarrollo y CI/CD

- **La versión de Node y de Alpine está fijada en el Dockerfile:** el contenedor ejecuta siempre lo mismo, sin depender del Node instalado en cada máquina.
- Desarrollo y producción comparten el mismo Dockerfile (solo cambia el target).
- La imagen que corre en Render es la misma que se construyó y público en CI, identificada por su tag SemVer; no se reconstruye en el servidor.

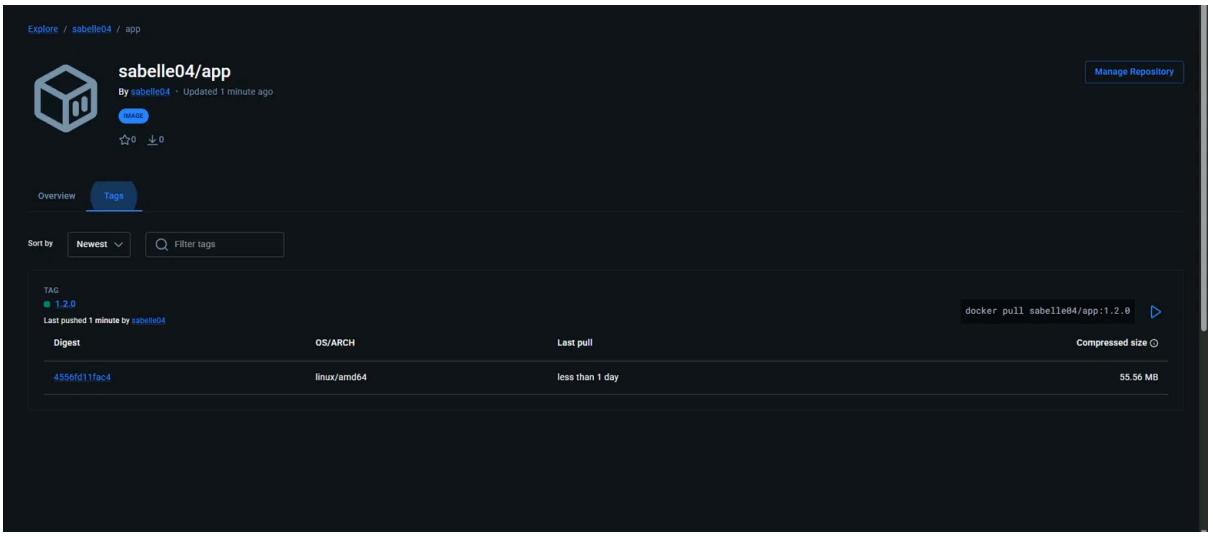
- npm ci instala exactamente lo que dice package-lock.json, y ***docker compose up --build*** levanta el entorno completo con un solo comando.

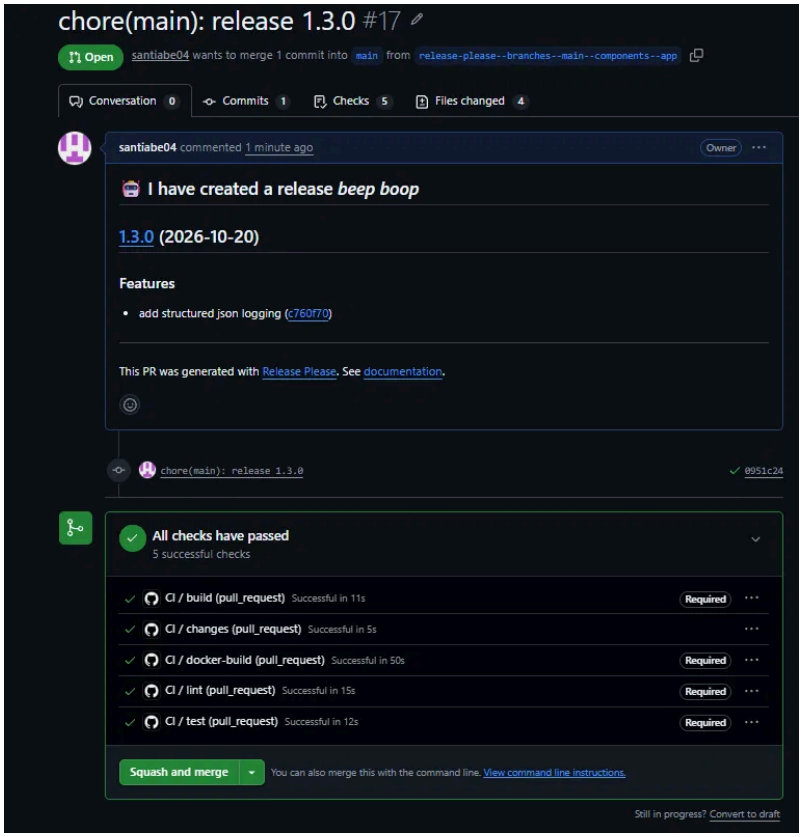
4.3. Segunda Forma: feedback rápido y Andon Cord

Puntos donde el pipeline "corta el cable"

Punto de corte	Qué frena
lint	Código que viola las reglas de ESLint
test	Cualquier prueba rota (incluidas las de documentación de la API)
build	Errores de compilación de TypeScript
docker-build	Dockerfile roto o dependencias faltantes en la imagen
Ruleset de main	Cualquier push directo o merge con checks en rojo
docker-publish / deploy	Solo corren si se creo un release; si falla el login o el push a Docker Hub, no se despliega

Los jobs son independientes y corren en paralelo: el feedback llega en minutos y se ve exactamente cuál fallo. El resultado se muestra directamente en el PR y en la pestaña Actions.






```
PS C:\Users\santi\Documents\TP_DEVOPS> git commit --allow-empty -m "chore: test direct push to main"
[main 962712c] chore: test direct push to main
PS C:\Users\santi\Documents\TP_DEVOPS> git push origin main
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 204 bytes | 204.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: error: GH013: Repository rule violations found for refs/heads/main.
remote: Review all repository rules at https://github.com/santiabe04/TP_DEVOPS/rules?ref=refs%2Fheads%2Fmain
remote:
remote: - 4 of 4 required status checks are expected.
remote:
remote: - Changes must be made through a pull request.
remote:
To github.com:santiabe04/TP_DEVOPS.git
 ! [remote rejected] main -> main (push declined due to repository rule violations)
error: failed to push some refs to 'github.com:santiabe04/TP_DEVOPS.git'
```

Visibilidad de fallos en producción

Síntoma	Herramienta	Señal
Respuestas HTTP 4xx / 5xx	Grafana Cloud	Métrica: paneles de errores (tasa y conteo de 4xx y 5xx)
Respuestas HTTP 4xx / 5xx	Logs de Render	Log: eventos JSON con ***level: "warn" / "error"*** y ***status_code***
Latencia alta	Grafana Cloud	Métrica: panel de latencia (p50 y p95) por histograma

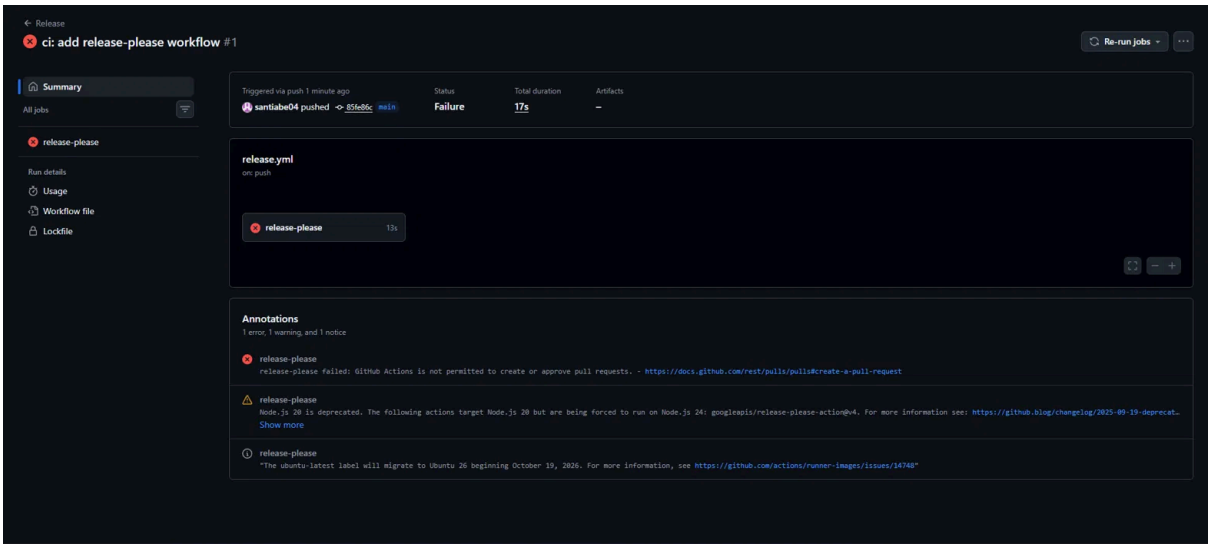
Servicio caído o suspendido	Grafana Cloud	Ausencia de nuevas series (el tráfico cae a cero)

4.4. Tercera Forma: aprendizaje y simulación de fallos

La Tercera Forma promueve la experimentación continua y el aprendizaje a partir de las fallas. Durante el desarrollo se registraron fallas reales que el propio pipeline o el monitoreo detectaron; cada una dejó una corrección y una lección.

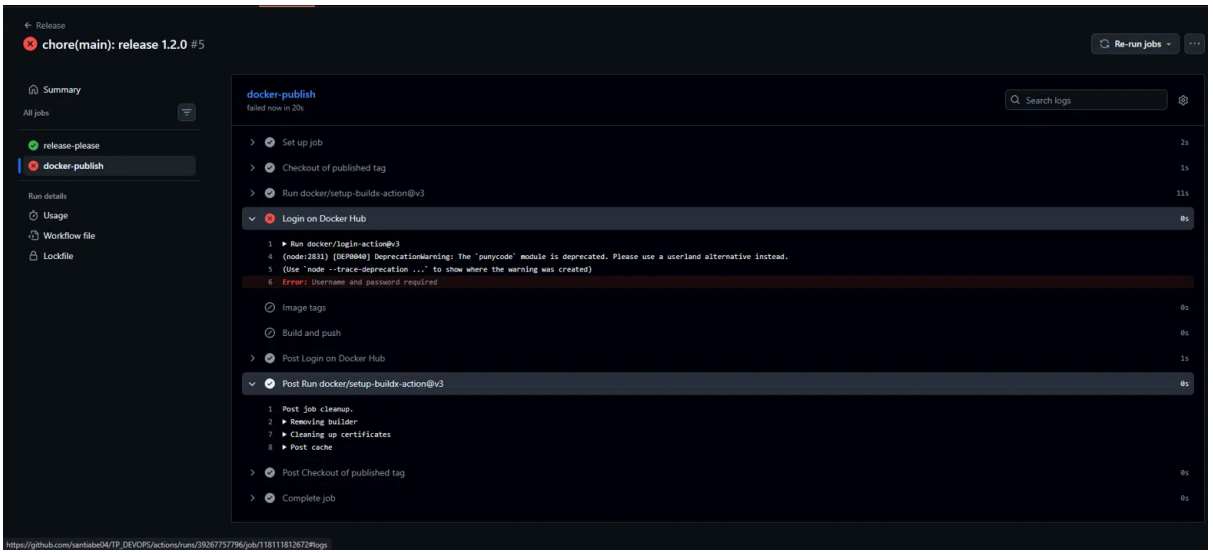
Falla 1: permisos de Actions (feedback en 17 segundos)

Al agregar release-please, el workflow fallo antes de abrir el PR de release con el error ***GitHub Actions is not permitted to create or approve pull requests***. La anotación del run lo mostró de inmediato. **Corrección:** habilitar en el repositorio los permisos de lectura/escritura y la creación de PRs por Actions, y relanzar el job. **Aprendizaje:** los prerrequisitos de plataforma también son parte del pipeline y se documentan



Falla 2: secrets faltantes (variable de entorno faltante)

En el primer release, el job docker-publish fallo en el login a Docker Hub con ***Username and password required***: los secrets no estaban cargados con el nombre esperado. Los pasos siguientes (etiquetado, build y push) quedaron omitidos. **Resultado del Andon Cord:** no se publicó ningún artefacto incompleto y el job deploy no se ejecutó. **Corrección:** cargar los secrets y relanzar solo los jobs fallidos, lo que publicó la versión 1.2.0



Falla 3: falla silenciosa de las métricas
Al instrumentar Open Telemetry la aplicación funcionaba sin errores pero Grafana no recibía datos, ni siquiera con un exportador de consola. La causa era el orden de inicialización: el histograma se creaba al importar el módulo, antes de que el SDK se iniciara, por lo que quedaba atado a un medidor no-op que descarta las mediciones sin avisar. **Corrección:** crear el histograma de forma lazy en la primera request. **Aprendizaje:** una falla puede no producir ningún error; hay que verificar el dato de punta a punta. El diagnóstico se hizo por capas: primero el middleware con un exportador de consola, luego las credenciales con una llamada directa al endpoint (respuesta 200) y finalmente la exportación real.

Falla 4: errores 5xx visibles en logs (A propósito)
Se generó tráfico con una mezcla de solicitudes exitosas (***/api/health***), inexistentes (***/api/no-existe***, 404) y a la ruta que falla a propósito (***/api/error***, 500) sobre el servicio desplegado en Render. Los eventos JSON salen con level ***info, warn y error*** según el resultado, y los 500 incluyen el stack trace, lo que permite pasar de la señal (métrica) a la causa (log).

5. Principios Lean: reducción de desperdicio

Lean busca maximizar el flujo de valor eliminando muda (desperdicio), muri (sobrecarga) y mura (inconsistencia). Poppendieck los traduce al software en siete desperdicios; los siguientes son los que la solución ataca directamente.

Desperdicio	Cómo se manifestaba	Cómo lo evita la solución
Defectos	Un bug se descubre en	Lint, pruebas y build en

	producción o en la revisión final y obliga a un trabajo urgente	cada PR; un test roto bloquea el merge antes de llegar a main (Build Integrity In)
Retrasos y esperas	Compilar, etiquetar, subir y desplegar a mano; esperar a que alguien lo haga	Pipeline automatizado de punta a punta; versión y tag generados automáticamente; checks en paralelo
Traspasos (handoffs)	Pérdida de conocimiento al pasar tareas entre Dev, QA y Ops	Un solo flujo con responsabilidad total: el mismo pipeline construye, verifica, pública, despliega y monitorea
Trabajo parcialmente hecho	Código sin probar, integrar o desplegar acumulandose en ramas largas	Ramas cortas y PRs pequeños que se integran y se despliegan en lotes chicos
Re-aprendizaje	"En mi maquina funciona y documentación ausente	Imagen con versiones fijadas, Dockerfile único, docker compose up reproducible, Swagger y changelog como documentación viva

Otros principios Lean donde aparecen: Deliver Fast (lotes pequeños, un cambio por PR), Amplify Learning (feedback corto en CI y dashboard), Build Integrity In (calidad integrada con tests automáticos y CI) y Optimize the Whole (el pipeline se diseño de punta a punta y no por pasos aislados).

6. Observabilidad

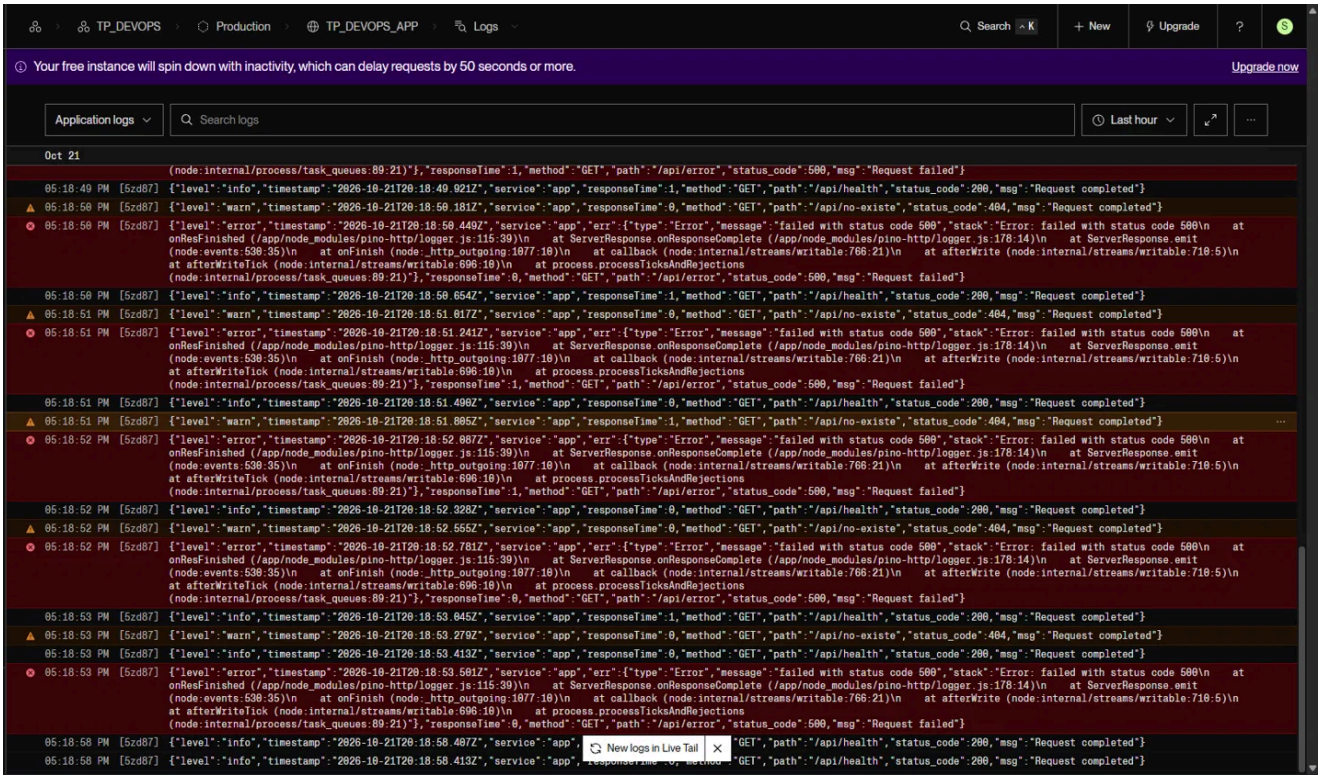
Este trabajo implementa métricas y logs. Las trazas distribuidas quedan fuera de alcance porque la API es un único servicio sin dependencias externas.

6.1. Logs estructurados

Cada request emite un evento JSON con pino y pino-http. El nivel depende del resultado: info (2xx/3xx), warn (4xx) y error (5xx). Incluye los campos requeridos por la consigna (timestamp, level, path, status_code) y además method, servicio y tiempo de respuesta.

{"level":"info","timestamp":"2026-10-21T18:57:27.116Z","service":"app","responseTime":2,"method":"GET","path":"/api/health","status_code":200,"msg":"Request completed"}

En tests el logger queda en modo silencioso para no ensuciar la salida. El formato JSON permite filtrar y agregar por campo en cualquier plataforma de logs



6.2. Plataforma de monitoreo y métricas

- **Plataforma:** Grafana Cloud, recibiendo métricas por OTLP/HTTP
- **Instrumentacion:** un middleware registra un histograma ***http.server.request.duration*** con tres atributos: method, ruta con patrón (por ejemplo ***/api/items/:id***, no la URL real, para no multiplicar las series) y código de estado.
- **Un solo histograma cubre las tres señales:** su conteo da tráfico y errores (filtrando por estado) y sus buckets dan los percentiles de latencia.
- **Configuración por entorno:** endpoint y credenciales se pasan por variables de entorno (OTEL_EXPORTER_OTLP_ENDPOINT, OTEL_EXPORTER_OTLP_HEADERS, OTEL_SERVICE_NAME). Sin endpoint, la telemetría queda desactivada (tests y desarrollo local), y el nombre de servicio permite distinguir en caso de querer hacer múltiples instalaciones.
- **Cierre ordenado:** al recibir la señal de terminación (Render la envía en cada deploy) se exportan las métricas pendientes antes de salir.

6.3. Dashboard y Golden Signals

El dashboard se construyó desde cero, sin plantillas, y está exportado en ***docs/grafana-dashboard.json***. El material define cuatro Golden Signals (latencia, tráfico, errores y saturación); la consigna pide las tres primeras, que son las que se miden a nivel HTTP.

Señal	Panel	Consulta
Tráfico	Time series (req/s)	***sum(rate(http_server_request_duration_seconds_count{service_name="app-tp-devops-render-prod"}[$__rate_interval]))***
Latencia	Time series p50 / p95	***histogram_quantile(0.50, sum by (le) (rate(http_server_request_duration_seconds_bucket{service_name="app-tp-devops-render-prod"}[$__rate_interval])))*** // ***histogram_quantile(0.95, sum by (le) (rate(http_server_request_duration_seconds_bucket{service_name="app-tp-devops-render-prod"}[$__rate_interval])))***
Errores (tasa)	Time series 4xx / 5xx	***sum(rate(http_server_request_duration_seconds_count{service_name="app-tp-devops-render-prod",http_response_status_code=~"4.."}[$__rate_interval])) or vector(0)*** // ***sum(rate(http_server_request_duration_seconds_count{service_name="app-tp-devops-render-prod",http_response_status_code=~"5.."}[$__rate_interval])) or vector(0)***
Errores (conteo)	Bar chart 4xx / 5xx	***sum(increase(http_server_request_duration_seconds_count{service_name="app-tp-devops-render-prod",http_response_status_code=~"4.."}[$__range])) or vector(0)*** //

		sum(increase(http_server_request_duration_seconds_count{service_name="app-tp-devops-render-prod",http_response_status_code=~"5.."}[$__range])) or vector(0)
Total de requests	Gauge / Stat	***sum(increase(http_server_request_duration_seconds_count{service_name="app-tp-devops-render-prod"}[$__range]))***



7. Conclusiones y mejoras

El trabajo logró un ciclo de vida completo y automatizado: un cambio nace en una rama corta, se verifica en un PR, se versiona y publica sin intervención manual (salvo la de lanzar un release), se despliega con la versión exacta y se observa en producción. Las fallas registradas durante el desarrollo mostraron que los mecanismos de stop-the-line y de monitoreo cumplen su función: cortaron el flujo en el punto correcto y dejaron evidencia para aprender de ellas.

Mejoras identificadas

- **Alertas:** Crear una regla de alerta en Grafana (por ejemplo, 5xx sostenidos) con notificación por correo, para pasar de "se ve en el dashboard" a "avisa al equipo" (alerta accionable basada en SLOs).
- **Seguridad en el pipeline (DevSecOps):** Agregar escaneo de vulnerabilidades de la imagen y de dependencias y detección de secretos al CI.
- **Saturación:** Instrumentar CPU, memoria y event loop para completar los cuatro Golden Signals.
- **Infraestructura como código:** Hoy el servicio de Render y el dashboard se configuraron desde la interfaz (el dashboard se exportó a JSON). Definir el servicio con un blueprint y versionar cerraría la brecha con la práctica de IaC.

- **Rollback:** Como el despliegue recibe la imagen con un tag exacto, volver a una versión anterior es relanzar el hook con el tag previo; automatizarlo como workflow permitirá implementar rollbacks automáticos ante errores.